Lab 2 — L'hexagone

Le domaine au centre, la technique en périphérie

Dr. El Hadji Bassirou TOURE
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop

Année universitaire 2026–2027

Résumé

Le Lab 1 a rangé SénResto en couches propres. Ce lab s'attaque au problème qui reste : le métier dépend toujours de la technique, trois étages plus bas, et demeure impossible à tester sans base de données. Vous allez **inverser cette dépendance** : le domaine définira ses besoins sous forme de **ports** (interfaces qui lui appartiennent), et la technique — JDBC, REST, notifications — deviendra un jeu d'**adaptateurs** interchangeables. La preuve finale tiendra en une démonstration : remplacer la base de données par une simple `Map` en mémoire **sans toucher une ligne du domaine**, et voir les tests du métier rendre leur verdict en deux secondes. Comme toujours : comportement de l'API inchangé, les 4 tests d'intégration restent verts du début à la fin.

Benefice

À l'issue de ce lab, vous saurez :
- appliquer le principe d'inversion de dépendances — et expliquer pourquoi tout se joue sur la *propriété de l'interface* ;
- structurer une application en **ports & adaptateurs** (architecture hexagonale) ;
- écrire des **tests unitaires du domaine** — sans Spring, sans BDD, sans HTTP ;
- démontrer le **swap d'adaptateur** : changer la technique sans toucher au métier.

Point d'attention

Prérequis avant la séance :
- votre dépôt au tag `lab2-depart` (= votre `lab1-final`, publié à l'ouverture de la séance) ;
- `./mvnw test` vert : `Tests run: 4, Failures: 0` ;
- l'application complète fonctionne (backend + frontend).

Si votre groupe n'a pas terminé le Lab 1 : `git checkout lab2-depart` sur le dépôt de référence — personne ne reste au bord de la route.

Table des matières

1 Le constat : essayez, vous n'y arriverez pas **3**
1.1 L'exercice impossible (15 minutes, en groupe) 3

2 Les concepts : inverser, c'est changer de propriétaire **3**
2.1 La cible : la nouvelle carte des paquetages 4

3 Etape 1 — Les ports : le domaine dicte ses besoins **5**

4 Etape 2 — Le grand déménagement **5**
4.1 Le domaine déménage — et perd ses annotations 6
4.2 Les repositories deviennent des adaptateurs 6
4.3 Spring assemble l'hexagone — dans config/, et nulle part ailleurs 6

5 Etape finale — Le swap, et la récompense **7**
5.1 Les adaptateurs in-memory 7
5.2 La récompense : le test impossible devient trivial 8

6 Recapitulatif : qui a fait quoi **9**

7 L'ADR du jour **9**

8 Livrable et criteres de reussite **9**

9 Vers le Lab 3 : la frontière qui manque **9**

1 Le constat : essayez, vous n'y arriverez pas

Intuition

Avant d'apprendre le remède, ressentez le mal. La direction de SénResto exige des tests du métier à chaque commit — rapides, donc unitaires. Prenez quinze minutes, en groupe, pour *essayer* d'écrire ce test. L'échec est au programme : c'est lui qui justifie tout le lab.

1.1 L'exercice impossible (15 minutes, en groupe)

Créez, dans `src/test/java`, la classe `sn.ucad.senresto.service.CommandeServiceTest` et tentez d'écrire un test de la règle des frais de livraison :

```
@Test
void lesFraisSuiventLeQuartier() {
    CommandeService service = new CommandeService(/* ...de quoi ? */);
    // Pour construire CommandeService, il faut : CommandeRepository,
    // RestaurantRepository, LivreurRepository, NotificationService.
    // Pour construire chaque repository, il faut : un JdbcTemplate.
    // Pour un JdbcTemplate, il faut : une DataSource... une BDD qui tourne.
}
```

Constatez et notez (vous en aurez besoin pour l'ADR) : pour tester une *multiplication et une addition*, il faut démarrer toute la pile technique. Supprimez ensuite ce fichier d'essai — nous le récrirons proprement en fin de lab, et il tiendra en dix lignes.

Point cle a retenir

Le Lab 1 a réglé le *désordre* ; il n'a pas réglé le **sens des flèches**. `CommandeService` → `Repository` → `JdbcTemplate` → BDD : tant que le métier pointe vers la technique, la technique dicte ses conditions au métier.

2 Les concepts : inverser, c'est changer de propriétaire

Concept fondamental

Principe d'inversion de dépendances (DIP). Le code de haut niveau (le métier) ne dépend pas du code de bas niveau (la technique) : tous deux dépendent d'**abstractions**. La subtilité décisive : ces interfaces appartiennent **au domaine**, sont écrites dans *son* paquetage et dans *son* vocabulaire. L'adaptateur technique les *implémente* — c'est donc lui qui importe le domaine, et la flèche de dépendance remonte.

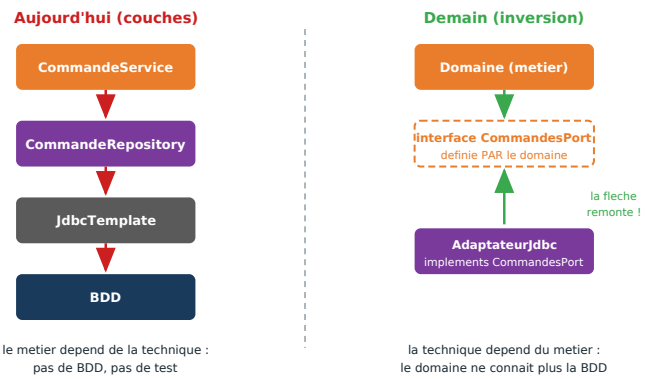


FIGURE 1 – L'inversion : la flèche entre métier et persistance change de sens.

Concept fondamental

Architecture hexagonale (ports & adaptateurs). Le domaine vit au centre, sans aucune dépendance vers l'extérieur — zéro Spring, zéro SQL, zéro HTTP. Sa frontière est faite de **ports** : les interfaces qu'il définit. Autour, les **adaptateurs** : entrants (REST traduit HTTP en appels du domaine) et sortants (JDBC, notifications implémentent les ports). Règle unique : **toutes les dépendances pointent vers le centre**.

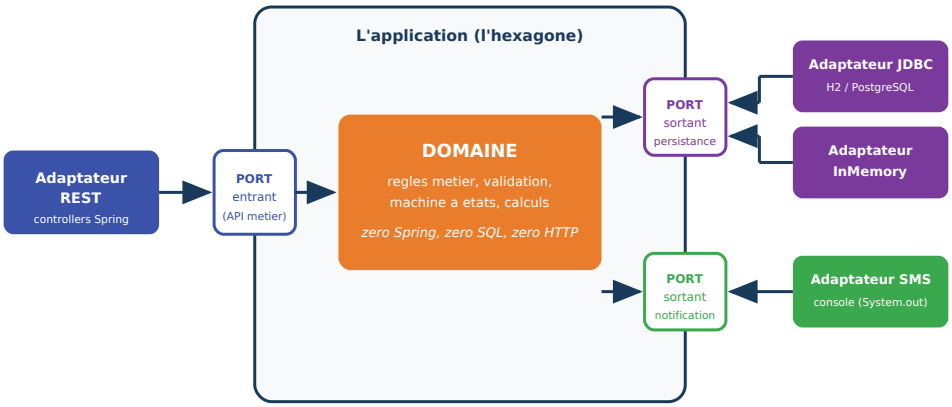


FIGURE 2 – La cible du lab : SénResto en ports & adaptateurs.

Exemple

Le test du vocabulaire, à appliquer à chaque port que vous écrirez : lisez-le à voix haute. `LivreursPort.premierDisponible()` — du métier, parfait. `executerRequeteSql(String sql)` — de la technique déguisée en port : refusé. Un port qui mentionne SQL, JSON ou HTTP est un aveu que la frontière fuit.

2.1 La cible : la nouvelle carte des paquetages

```
sn/ucad/senresto/
|-- SenRestoApplication.java
|-- domaine/                        <- LE CENTRE : zero import Spring
|   |-- CommandeMetier.java              (ex-CommandeService, sans annotations)
|   |-- RestaurantMetier.java
|   |-- LivreurMetier.java
|   |-- QuartiersDakar.java
|   |-- CalculateurEta.java
|   |-- ValidationException.java  IntrouvableException.java  ConflitException.java
|   \-- port/                       <- les interfaces DU domaine
|       |-- CommandesPort.java
|       |-- RestaurantsPort.java
|       |-- LivreursPort.java
|       \-- NotificationsPort.java
|-- adaptateur/
|   |-- rest/                       <- adaptateur ENTRANT
|   |   |-- SenRestoController.java   GestionnaireErreurs.java
|   |   \-- dto/  (CommandeRequete, ChangementStatutRequete)
|   |-- persistance/                <- adaptateurs SORTANTS (JDBC)
|   |   |-- AdaptateurCommandesJdbc.java      implements CommandesPort
|   |   |-- AdaptateurRestaurantsJdbc.java    implements RestaurantsPort
|   |   \-- AdaptateurLivreursJdbc.java       implements LivreursPort
|   \-- notification/
|       \-- AdaptateurSmsConsole.java         implements NotificationsPort
\-- config/
    \-- ConfigurationDomaine.java   <- Spring instancie le domaine ICI
```

Point d'attention

La règle sacrée du lab, vérifiable mécaniquement à tout moment :

```
grep -r "import org.springframework"
backend/src/main/java/sn/ucad/senresto/domaine/
```

doit rendre **zéro ligne**, du premier commit au dernier. Le domaine ne connaît ni Spring, ni JDBC, ni HTTP — c'est toute sa valeur.

3 Etape 1 — Les ports : le domaine dicte ses besoins

Qui pilote cette section ?

L'Huissier pilote : il lit les trois repositories du Lab 1 et en extrait les signatures *dont le métier a réellement besoin*. **Le Notaire** vérifie chaque signature contre son `INVENTAIRE-SQL.md` du Lab 1. **Les autres** préparent leurs branches d'étape 2. **Le Doyen** arbitre le vocabulaire des ports.

Créez le paquetage `domaine/port` et les quatre interfaces. Le patron — remarquez : **aucun import**, le vocabulaire du métier, rien de plus que le besoin :

```java
package sn.ucad.senresto.domaine.port;

import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Ce dont le metier a besoin pour les livreurs -- rien de plus. */
public interface LivreursPort {
    List<Map<String, Object>> tous();
    Optional<Map<String, Object>> premierDisponible();
    void reserver(long livreurId);
    void liberer(long livreurId);
}
```

Sur le même modèle : `CommandesPort` (inserer, majNumero, majStatut, majConfirmation, supprimer, toutes, parIdComplet) ; `RestaurantsPort` (tous, parId, platsDuRestaurant, platParId) ; et `NotificationsPort` (envoyerSms). Les signatures existent déjà dans vos repositories : vous ne créez pas du code, vous déclarez *à qui il appartient*.

Tache a realiser

À réaliser — étape 1 (branche `feature/ports`) : les quatre interfaces dans `domaine/port`, relues à voix haute (test du vocabulaire), compilation verte. C'est une étape courte et décisive : tout le reste du lab s'appuie dessus.

Jalon Git

Compilation verte, vocabulaire validé par le Doyen :

```
git tag lab2-etape1 && git push origin lab2-etape1
```

4 Etape 2 — Le grand déménagement

Qui pilote cette section ?

L'Huissier déménage le métier dans `domaine/`. **Le Notaire** transforme les repositories en adaptateurs (`implements`). **Le Rédacteur** déplace REST et DTOs dans `adaptateur/rest`. **L'Appariteur** écrit `ConfigurationDomaine` et l'adaptateur SMS. **Le Doyen** merge dans l'ordre : ports → domaine → adaptateurs → config.

4.1 Le domaine déménage — et perd ses annotations

`CommandeService` devient `domaine/CommandeMetier` : mêmes méthodes, mêmes règles, mêmes messages — mais **plus aucune annotation Spring**, et des dépendances déclarées *sur les ports* :

```
package sn.ucad.senresto.domaine;

import sn.ucad.senresto.domaine.port.*;
// AUCUN import org.springframework : le domaine est du Java pur.

public class CommandeMetier {

    private final CommandesPort commandes;
    private final RestaurantsPort restaurants;
    private final LivreursPort livreurs;
    private final NotificationsPort notifications;

    public CommandeMetier(CommandesPort commandes, RestaurantsPort restaurants,
                          LivreursPort livreurs, NotificationsPort notifications) {
        this.commandes = commandes;
        this.restaurants = restaurants;
        this.livreurs = livreurs;
        this.notifications = notifications;
    }
    // creer(), confirmer(), changerStatut(), supprimer(), lister(), detail() :
    // le corps des methodes du Lab 1, inchange -- seuls les TYPES des champs
    // ont change : des ports, plus des repositories.
}
```

Point d'attention

Question d'équipe (à trancher, puis à consigner dans l'ADR) : où vivent les DTOs `CommandeRequete` et `ChangementStatutRequete` ? S'ils restent dans l'adaptateur REST, le domaine ne peut pas les recevoir en paramètre sans dépendre de l'adaptateur — interdit. Deux options propres : les déplacer dans le **domaine** (ce sont des demandes métier, choix recommandé ici), ou faire traduire l'adaptateur REST vers des types du domaine. Choisissez en connaissance de cause.

4.2 Les repositories deviennent des adaptateurs

Pour le Notaire, l'étape est presque cérémonielle — le SQL ne change pas d'une virgule, seule l'*appartenance* change :

```
package sn.ucad.senresto.adaptateur.persistance;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import sn.ucad.senresto.domaine.port.LivreursPort;
// L'adaptateur importe le domaine : la fleche pointe vers le centre.

@Repository
public class AdaptateurLivreursJdbc implements LivreursPort {
    // ... le contenu exact de LivreurRepository (Lab 1), avec @Override ...
}
```

4.3 Spring assemble l'hexagone — dans config/, et nulle part ailleurs

Le domaine n'ayant plus d'annotations, quelqu'un doit l'instancier. C'est l'unique rôle de la classe `ConfigurationDomaine`, seul endroit où Spring et le domaine se rencontrent :

```
package sn.ucad.senresto.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import sn.ucad.senresto.domaine.*;
import sn.ucad.senresto.domaine.port.*;

@Configuration
public class ConfigurationDomaine {

    @Bean
    public CommandeMetier commandeMetier(CommandesPort commandes,
                                         RestaurantsPort restaurants,
                                         LivreursPort livreurs,
                                         NotificationsPort notifications) {
        return new CommandeMetier(commandes, restaurants, livreurs, notifications);
    }

    @Bean
```

```
    public RestaurantMetier restaurantMetier(RestaurantsPort restaurants) {
        return new RestaurantMetier(restaurants);
    }

    @Bean
    public LivreurMetier livreurMetier(LivreursPort livreurs) {
        return new LivreurMetier(livreurs);
    }
}
```

Spring injecte automatiquement chaque adaptateur `@Repository` là où son *interface* de port est attendue : l'assemblage est déclaratif. Le controller, lui, change trois types d'import (`CommandeMetier` au lieu de `CommandeService`) et rien d'autre.

Tache a realiser

À réaliser — étape 2 :

- L'Huissier (`feature/domaine`) : `CommandeMetier`, `RestaurantMetier`, `LivreurMetier`, exceptions, `QuartiersDakar`, `CalculateurEta` déménagés dans `domaine/` — et le `grep` sacré qui rend zéro ;
- Le Notaire (`feature/adaptateurs-jdbc`) : les trois `AdaptateurXxxJdbc implements XxxPort` ;
- Le Rédacteur (`feature/adaptateur-rest`) : controller, gestionnaire d'erreurs et DTOs dans `adaptateur/rest` (ou DTOs dans le domaine selon la décision d'équipe) ;
- L'Appariteur (`feature/config`) : `AdaptateurSmsConsole` et `ConfigurationDomaine` ;
- Le Doyen : merges dans l'ordre, `./mvnw test` après chacun.

Jalon Git

`./mvnw test` vert (4/4), parcours frontend complet, grep sacré à zéro :

```
git tag lab2-etape2 && git push origin lab2-etape2
```

5 Etape finale — Le swap, et la récompense

Qui pilote cette section ?

L'Appariteur pilote : les adaptateurs in-memory et les nouveaux tests sont son chef-d'œuvre du jour. **L'Huissier** valide que le domaine n'a pas bougé d'un caractère. **Les autres** écrivent chacun un test unitaire du domaine.

5.1 Les adaptateurs in-memory

Dans `src/test/java`, paquetage `sn.ucad.senresto.domaine.enmemoire` : quatre classes qui implémentent les ports avec de simples structures Java. Le patron :

```
/** Un "stockage" de commandes : une Map et un compteur. C'est tout. */
public class CommandesEnMemoire implements CommandesPort {

    private final Map<Long, Map<String, Object>> donnees = new LinkedHashMap<>();
    private long prochainId = 1;

    @Override
    public long inserer(String clientNom, String clientTelephone, String clientQuartier,
                        long platId, int quantite, int fraisLivraison, int montantTotal) {
        long id = prochainId++;
        Map<String, Object> c = new LinkedHashMap<>();
        c.put("id", id); c.put("numero", "CMD-TEMP");
        c.put("client_nom", clientNom); c.put("client_telephone", clientTelephone);
        c.put("client_quartier", clientQuartier); c.put("plat_id", platId);
        c.put("quantite", quantite); c.put("frais_livraison", fraisLivraison);
        c.put("montant_total", montantTotal); c.put("statut", "EN_ATTENTE");
        donnees.put(id, c);
        return id;
    }
    // ... les autres methodes : lectures et mises a jour de la Map ...
}
```

5.2 La récompense : le test impossible devient trivial

Récrivez maintenant le test de la section 1 — celui qui était impossible :

```
class CommandeMetierTest {

    private CommandeMetier domaine() {
        return new CommandeMetier(new CommandesEnMemoire(),
                new RestaurantsEnMemoire(), new LivreursEnMemoire(),
                new NotificationsEnMemoire());
    }

    @Test
    void lesFraisDeLivraisonSuiventLeQuartier() {
        var commande = domaine().creer(new CommandeRequete(
                "Awa Ba", "771112233", "Ouakam", 1L, 2));
        assertEquals(2500 * 2 + 1500, commande.get("montant_total"));
    }

    @Test
    void confirmerAssigneUnLivreurEtCalculeUnEta() { /* a vous */ }

    @Test
    void uneTransitionInterditeEstRefusee() { /* a vous */ }
}
```

Lancez, et savourez le chronomètre : `./mvnw test` exécute désormais les 4 tests d'intégration *et* vos tests unitaires — et ces derniers rendent leur verdict en **quelques secondes, sans base de données**. Faites la démonstration inverse pour la beauté du geste : coupez Docker, éteignez tout — les tests du domaine passent toujours.

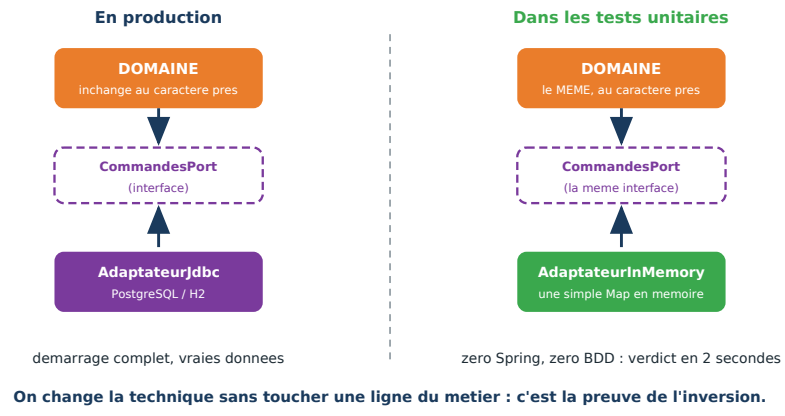


FIGURE 3 – Le swap : même domaine, technique interchangeable — la preuve de l'inversion.

Point cle a retenir

Vous venez de constituer la **pyramide des tests** : nombreux, rapides et unitaires au centre (le domaine) ; rares, lents et intégrés en périphérie (le branchement complet). Les deux familles sont complémentaires — aucune ne remplace l'autre.

Tache a realiser

À réaliser — étape finale :

- L'Appariteur (`feature/enmemoire`) : les quatre adaptateurs in-memory dans `src/test/java` ;
- Chaque membre : **au moins un test unitaire** du domaine signé de sa main (frais, ETA, transitions, validation, quantités...) ;
- L'Huissier : vérification que `domaine/` est identique entre `lab2-etape2` et `lab2-final` (`git diff lab2-etape2 - backend/src/main/java/sn/ucad/senresto/domaine` : vide) ;
- Tous : `./mvnw test` — intégration 4/4 *et* unitaires verts.

Jalon Git

Le verdict final — tests d'intégration ET unitaires verts, grep sacré à zéro, frontend intact :

```
git tag lab2-final && git push origin lab2-final
```

6 Recapitulatif : qui a fait quoi

Rôle	Étape 1	Étape 2	Étape finale
Le Doyen	arbitrage du vocabulaire	merges ordonnés, tests	tag final, ADR
L'Huissier	extraction des signatures	déménagement du domaine, grep sacré	domaine gelé (diff vide)
Le Rédacteur	relecture des ports	adaptateur REST, décision DTOs	un test unitaire signé
Le Notaire	vérif. contre l'inventaire SQL	les trois adaptateurs JDBC	un test unitaire signé
L'Appariteur	branches préparées	`ConfigurationDomaine`, SMS	in-memory + tests unitaires

7 L'ADR du jour

L'**Architecte du jour** rédige et signe l'**ADR n°2 : « Pourquoi inverser les dépendances ? »** (gabarit habituel, `docs/adr/adr-002-inversion.md`, avant le tag final). Matière à alternatives sérieuses : rester en couches et tester le métier avec une BDD embarquée dédiée aux tests ? Utiliser des mocks (Mockito) sur les repositories sans inverser quoi que ce soit ? Matière à conséquences négatives : le coût des interfaces, l'indirection, la `ConfigurationDomaine` à maintenir, la discipline du vocabulaire. Et une exigence de contexte : citer *votre* expérience de la section 1 — le test impossible.

8 Livrable et criteres de reussite

Critère	Vérification
Les 4 tags du lab présents sur le dépôt	`git tag`
Tests d'intégration ET unitaires verts sur `lab2-final`	`./mvnw test`
Zéro import Spring dans `domaine/`	le grep sacré
Les quatre ports dans `domaine/port`, vocabulaire métier	relecture
Le domaine identique entre etape2 et final	`git diff` vide
Au moins un test unitaire signé par membre	historique Git
Frontend intact, parcours complet fonctionnel	démonstration
ADR n°2 signé dans `docs/adr/`	relecture

9 Vers le Lab 3 : la frontière qui manque

Votre hexagone protège le métier de la technique. Mais regardez *à l'intérieur* du domaine : commandes, restaurants, livreurs, notifications — et bientôt le paiement mobile money — vivent tous dans le *même* hexagone, libres de s'appeler les uns les autres. Quand SénResto grandira, qui empêchera le code du Catalogue de fouiller dans les données du Paiement ? Il manque des frontières *entre les métiers* — pas seulement entre le métier et la technique. C'est le sujet du Lab 3 : le **monolithe modulaire**, avec Spring Modulith en gardien automatique des frontières. Votre `lab2-final` en sera le point de départ.

Fin du Lab 2 — Dr. El Hadji Bassirou TOURE — DMI/FST/UCAD